



ESCUELA DE  
INGENIERÍA EN CIENCIAS Y SISTEMAS  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



|                 |                    |
|-----------------|--------------------|
| Día, Fecha:     | JUEVES, 30/07/2026 |
| Hora de inicio: | 17:20              |

## ANÁLISIS Y DISEÑO DE SISTEMAS 2 SECCIÓN B

Escuela de Ingeniería en Ciencias y Sistemas

ABDIEL FERNANDO JOSÉ OTZOY OTZÍN

- Presentación de la clase
- Competencias que se desarrollarán
- Valor de la semana: Responsabilidad
- Resolución de dudas sobre la Práctica 1
- Introducción a los patrones de diseño
- Patrones de diseño creacionales
- Comparación de los patrones
- Video de refuerzo
- Conclusiones y dudas finales
- Recordatorio del foro semanal

**Competencia principal:** aplicar patrones creacionales para mejorar la creación de objetos, reducir acoplamiento y hacer el sistema más mantenible.

## Al final de la clase podrás...

- Identificar problemas de creación de objetos.
- Diferenciar los cinco patrones creacionales.
- Elegir un patrón según el escenario.
- Reconocer ventajas, límites y riesgos.
- Aplicar un ejemplo sencillo en TypeScript.

# Responsabilidad

Cumplir compromisos, comunicar riesgos a tiempo y asumir las consecuencias de las propias decisiones.

## En desarrollo de software se ve así...

- Cumplir tareas y fechas establecidas.
- Comunicar bloqueos o dificultades a tiempo.
- Revisar y documentar el código antes de entregarlo.
- Elegir patrones con criterio, no por costumbre.

**Reflexión:** ¿cómo afecta al equipo que alguien no cumpla con lo acordado?

- Explicar qué es un patrón de diseño y qué problema busca resolver.
- Reconocer la clasificación general: creacionales, estructurales y de comportamiento.
- Identificar las características de los patrones creacionales.
- Explicar qué problema resuelve cada patrón estudiado.
- Diferenciar Factory Method, Abstract Factory, Singleton, Builder y Prototype.
- Seleccionar un patrón apropiado según un problema de diseño.
- Interpretar diagramas UML básicos y ejemplos sencillos de implementación.
- Identificar ventajas, desventajas y riesgos de cada patrón.
- Justificar la elección de un patrón dentro de un escenario de software.

## Criterio de logro

Puedes mirar un escenario y decir:  
problema → patrón → razón → riesgo. Si  
falta una de las cuatro partes, todavía hay  
algo que repasar.

- Comprobación continua: cada patrón termina con una pregunta breve.

# Dudas de la Práctica 1

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

## Espacio para preguntas sobre la Práctica 1

---

*Se atienden dudas puntuales antes de iniciar el tema nuevo.*

```
ServicioPedido.ts

class ServicioPedido {
  procesar(tipo: string) {
    let pago;
    if (tipo === "tarjeta") {
      pago = new PagoTarjeta();
    } else if (tipo === "paypal") {
      pago = new PagoPayPal();
    } else {
      pago = new PagoEfectivo();
    }
    pago.cobrar();
  }
}
```

Cada nuevo método de pago obliga a abrir y modificar ServicioPedido.

### Qué está pasando

- La clase cliente conoce clases concretas: PagoTarjeta, PagoPayPal, PagoEfectivo.
- La decisión de creación queda mezclada con la lógica de cobro.
- Agregar un tipo nuevo exige tocar código que ya funcionaba.
- Las pruebas se vuelven difíciles porque la dependencia nace dentro del método.

**Pregunta orientadora:** ¿quién debería decidir qué objeto de pago se crea si mañana aparece “PagoConCripto”?

# ¿Qué es un patrón de diseño?

**Definición corta:** un patrón de diseño describe una forma probada de organizar clases y objetos para resolver un problema que aparece una y otra vez. No es código para copiar; es una guía con nombres, responsabilidades y consecuencias.

**¿Por qué surgieron?** Para capturar experiencia de diseño, dar vocabulario común al equipo y evitar resolver desde cero problemas ya estudiados.

■ Beneficio: comunicación y reutilización de decisiones. ■ Riesgo: aplicarlo sin necesidad aumenta complejidad.

## Límites rápidos

| Concepto     | Qué es                                  | No confundir con patrón                          |
|--------------|-----------------------------------------|--------------------------------------------------|
| Algoritmo    | Pasos para calcular un resultado        | No organiza responsabilidades entre clases       |
| Librería     | Código listo para llamar                | No obliga a una estructura de diseño             |
| Framework    | Marco que dirige parte de la aplicación | Es más grande y opinado que un patrón            |
| Arquitectura | Decisiones de alto nivel del sistema    | El patrón suele actuar a nivel de clases/objetos |

Un patrón no reemplaza juicio de diseño: se elige por el problema, no por moda.



# Clasificación de patrones

FUNDAMENTOS

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

## Creacionales

Controlan cómo nacen los objetos: quién los crea, con qué configuración y cuántas instancias existen.

## Estructurales

Explican cómo componer clases y objetos para formar estructuras flexibles.

## Comportamiento

Definen cómo los objetos se comunican y reparten responsabilidades.

**Foco de hoy:** los patrones creacionales separan “usar un objeto” de “decidir cómo se crea”. Esa separación permite cambiar la creación sin reescribir la lógica del sistema.

Ejemplos del día: **Factory Method, Abstract Factory, Singleton, Builder y Prototype.**

FUNDAMENTOS



**Clase:** molde o definición.

**Objeto / instancia:** algo creado desde una clase.

**Constructor:** método que inicializa un objeto.

**Dependencia:** una clase necesita conocer a otra.

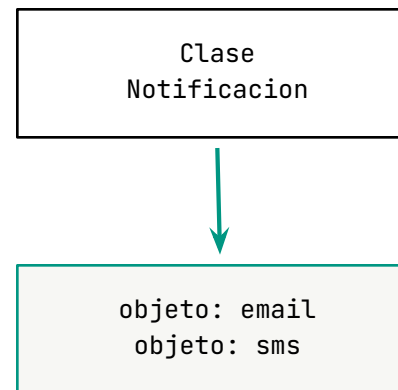
**Acoplamiento:** cuánto afecta un cambio entre clases.

**Configuración:** datos que definen una variante.

**Familia de objetos:** objetos relacionados que deben ser compatibles.

**Instanciación:** el acto de crear una instancia.

### Lectura rápida



- La clase define.
- La instancia existe en memoria.
- El constructor configura el estado inicial.

**Idea fuerza:** si la creación cambia seguido, conviene moverla fuera del código que solo quiere usar el objeto.

# Mapa de decisión

| Si el problema es...               | Pregunta clave                         | Patrón probable         | Señal de alerta                  |
|------------------------------------|----------------------------------------|-------------------------|----------------------------------|
| Elegir una variante de un producto | ¿Qué subclase decide el tipo exacto?   | <b>Factory Method</b>   | Muchos if con new                |
| Crear familias compatibles         | ¿Botón + ventana deben combinar?       | <b>Abstract Factory</b> | Mezclar piezas incompatibles     |
| Controlar cantidad de instancias   | ¿Debe existir solo una?                | <b>Singleton</b>        | Estado global disfrazado         |
| Construir con muchas opciones      | ¿Hay parámetros opcionales en cascada? | <b>Builder</b>          | Constructor telescópico          |
| Reusar una configuración existente | ¿Copiar es más barato que construir?   | <b>Prototype</b>        | Referencias internas compartidas |

**Advertencia:** no existe un patrón universalmente mejor. Se selecciona por contexto, costo de cambio y riesgo de complejidad.

# Factory Method: idea y problema

**En una oración:** define un método para crear objetos, pero permite que las subclases decidan qué clase concreta instanciar.

**Analogía:** el mesero no cocina; entrega la orden a la cocina. La cocina especializada decide cómo preparar ese plato.

**Constructor directo:** el cliente conoce la clase. **Método fábrica:** el cliente pide el producto a través de una abstracción.

```
sin_patron.ts
// Sin patrón: decisión incrustada
if (canal === "email") {
  n = new EmailNotificacion();
} else {
  n = new SmsNotificacion();
}
n.enviar(msg);
```

## Problema

- El cliente queda acoplado a EmailNotificacion y SmsNotificacion.
- Cada canal nuevo obliga a modificar el mismo condicional.
- La creación y el envío quedan mezclados.

**Anticipa:** ¿qué clase podría especializarse en crear cada notificación?

# Factory Method: solución y UML

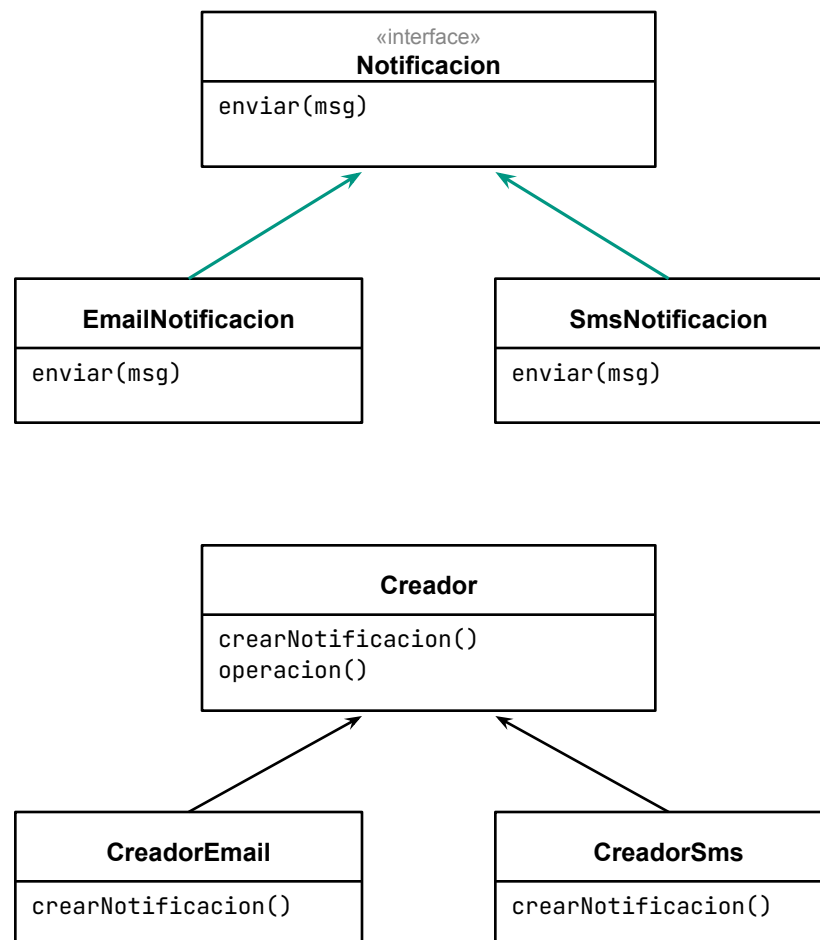
- 1. El cliente trabaja con la interfaz Notificacion.
- 2. Llama a un Creador sin conocer la clase concreta.
- 3. Cada CreadorConcreto implementa crearNotificacion().
- 4. El producto creado se usa por su interfaz, no por su tipo.

## Participantes

- **Producto:** Notificacion.
- **Producto concreto:** Email, SMS.
- **Creador:** declara crearNotificacion().
- **Creador concreto:** decide la instancia.

Diagrama guiado: leer primero la interfaz, luego los creadores

01 · FACTORY METHOD



factory\_method.ts

```
interface Notificacion { enviar(msg: string): void }  
class EmailNotificacion implements Notificacion {  
  enviar(msg: string) { /* SMTP */ }  
}  
abstract class Creador {  
  abstract crear(): Notificacion;  
  avisar(msg: string) { this.crear().enviar(msg); }  
}  
class CreadorEmail extends Creador {  
  crear() { return new EmailNotificacion(); }  
}
```

Lectura línea clave: avisar() no sabe si enviará por email o SMS; solo exige que exista enviar().

## Evaluación rápida

- **Ventaja:** agregar un canal no modifica el cliente.
- **Costo:** más clases y una dirección indirecta.
- **Usar cuando:** las variantes crecen o cambian seguido.
- **Evitar cuando:** solo hay una variante estable.
- **Error frecuente:** convertir cada new en fábrica sin necesidad.

**Comprobación:** ¿qué cambiaría si mañana se agrega WhatsAppNotificacion?

# Abstract Factory: idea y problema

**En una oración:** proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.

**Analogía:** comprar un juego de muebles por estilo: silla, mesa y lámpara deben combinar; no mezclas piezas de estilos incompatibles.

**Familia de objetos:** conjunto de productos relacionados que deben ser compatibles. Si el sistema corre en Windows, botón, menú y ventana deberían venir de la misma familia Windows.

```
mezcla_incompatible.ts

// Riesgo de mezcla incompatible
const boton = new BotonWindows();
const ventana = new VentanaLinux();
boton.dibujar();
ventana.abrir();
```

## Problema

- El cliente puede combinar productos de familias distintas.
- La compatibilidad queda en manos de quien recuerda elegir bien.
- Cambiar de plataforma exige buscar todos los new dispersos.

**Anticipa:** ¿cómo garantizar que botón y ventana siempre pertenezcan a la misma familia?

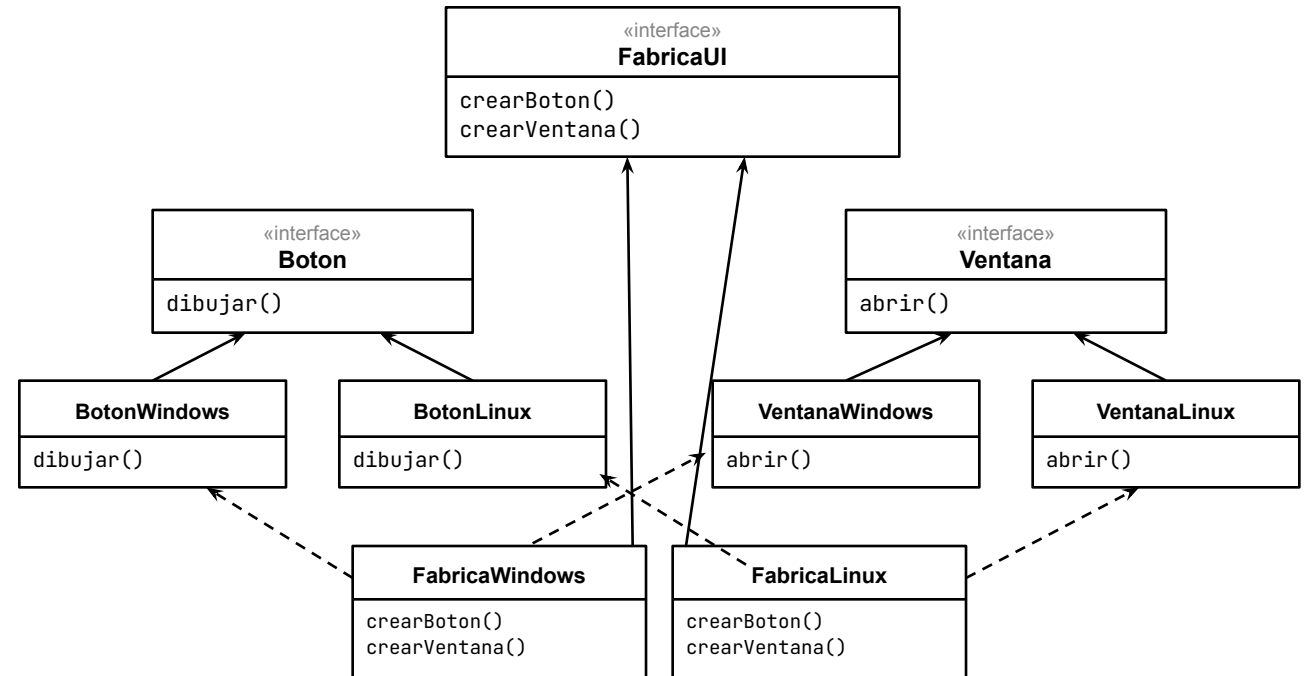
- 1. El cliente recibe una FabricaUI abstracta.
- 2. Pide crearBoton() y crearVentana() sin saber la plataforma.
- 3. Una fábrica concreta devuelve productos de una misma familia.
- 4. Cambiar de familia cambia la fábrica, no la lógica cliente.

### Participantes

- **Fábrica abstracta:** FabricaUI.
- **Fábricas concretas:** Windows, Linux.
- **Productos abstractos:** Boton, Ventana.
- **Productos concretos:** BotonWindows, BotonLinux, VentanaWindows, VentanaLinux.

Cada interfaz tiene versión Windows y Linux

02 · ABSTRACT FACTORY



Flechas punteadas: la fábrica crea el producto. Cada método de la fábrica puede verse como un Factory Method; el patrón agrega la idea de familia completa.



abstract\_factory.ts

```
interface Boton { dibujar(): void }
interface Ventana { abrir(): void }
class BotonWindows implements Boton { dibujar() { /* estilo Windows */ } }
class BotonLinux implements Boton { dibujar() { /* estilo Linux */ } }
class VentanaWindows implements Ventana { abrir() { /* estilo Windows */ } }
class VentanaLinux implements Ventana { abrir() { /* estilo Linux */ } }
interface FabricaUI {
  crearBoton(): Boton;
  crearVentana(): Ventana;
}
class FabricaWindows implements FabricaUI {
  crearBoton() { return new BotonWindows(); }
  crearVentana() { return new VentanaWindows(); }
}
class FabricaLinux implements FabricaUI {
  crearBoton() { return new BotonLinux(); }
  crearVentana() { return new VentanaLinux(); }
}
function render(f: FabricaUI) {
  f.crearBoton().dibujar();
  f.crearVentana().abrir();
}
```

Resultado esperado: render(new FabricaLinux()) dibuja la familia Linux completa; el cliente no cambia.

## 02 · ABSTRACT FACTORY

## Evaluación y límite

- **Ventaja:** compatibilidad de familia garantizada por construcción.
- **Costo:** agregar un nuevo producto obliga a tocar todas las fábricas.
- **Usar cuando:** las combinaciones incorrectas son un riesgo real.
- **Evitar cuando:** solo existe un producto aislado.
- **FM vs AF:** FM crea un producto; AF crea una familia relacionada.

**Comprobación:** ¿por qué mezclar BotonWindows con VentanaLinux es un error de diseño y no solo de estilo?

**En una oración:** asegura que una clase tenga una sola instancia y ofrece un punto global de acceso a ella.

**Analogía:** una sola impresora compartida en la oficina: todos la usan, pero no cada persona instala una impresora distinta.

**Ejemplos habituales:** configuración de la aplicación, registro de eventos, administración de un recurso costoso o caché compartida. La clave es que el recurso realmente deba ser único.

03 · SINGLETON

config\_problema.ts

```
// Problema: configuraciones inconsistentes
const c1 = new Config();
const c2 = new Config();
c1.modos = "debug";
console.log(c2.modos); // no refleja c1
```

## Problema

- Dos objetos Config pueden divergir.
- No hay un punto claro para leer el estado compartido.
- La tentación es resolverlo con una variable global sin control.

**Anticipa:** ¿quién debería impedir que existan dos instancias de Config?

```
singleton.ts

class Config {
  private static instancia?: Config;
  private constructor(public readonly modo: string) {}
  static get(): Config {
    if (!Config.instancia) {
      Config.instancia = new Config("prod");
    }
    return Config.instancia;
  }
}

const a = Config.get();
const b = Config.get();
console.log(a === b); // true
```

Secuencia: get() revisa si ya existe; si no, la crea una vez; después devuelve la misma referencia.

## 03 · SINGLETON

## Funcionamiento paso a paso

- 1. Constructor privado: nadie fuera puede hacer new.
- 2. Atributo estático: guarda la única instancia.
- 3. Método get(): crea solo si aún no existe.
- 4. Clientes reciben el mismo objeto y comparten estado.

**Advertencia:** en entornos concurrentes o con múltiples cargadores de módulos, “una instancia” puede ser más difícil de garantizar de lo que parece.

## Cuándo podría ser apropiado

- Existe un recurso único por naturaleza: configuración global, registro central, pool costoso.
- Se necesita un punto de acceso claro y controlado.
- El estado compartido es pequeño, estable y fácil de reiniciar en pruebas.

## Cuándo debería evitarse

- Solo se quiere una variable global conveniente.
- Las pruebas necesitan estados distintos en paralelo.
- La dependencia queda oculta dentro de muchos métodos.

## Riesgos que hay que nombrar

03 · SINGLETON

- **Estado global:** cambios invisibles afectan partes lejanas.
- **Pruebas:** una instancia residual contamina el siguiente caso.
- **Concurrencia:** dos hilos podrían intentar crearla a la vez.
- **Acoplamiento:** las clases dejan de recibir lo que necesitan.

**Comprobación:** si tu prueba unitaria necesita “otra configuración”, ¿Singleton sigue siendo buena idea?

**En una oración:** separa la construcción de un objeto complejo de su representación final, permitiendo crearlo paso a paso.

**Analogía:** armar un pedido: primero eliges base, luego proteína, extras y empaque; no dices todo en una sola frase confusa.

**Clave práctica:** distingue parámetros obligatorios de opcionales. Los obligatorios pueden ir al inicio; los opcionales se agregan con métodos claros y encadenables.

04 · BUILDER

● ● ● reporte\_telescopico.ts

```
// Constructor telescópico  
new Reporte("ventas", true, false, true,  
  null, "PDF", 30, false, "admin");
```

## Problema

- No se entiende qué significa cada true, false o null.
- Es fácil intercambiar parámetros del mismo tipo.
- Agregar una opción obliga a revisar todas las llamadas.

**Anticipa:** ¿cómo harías legible un reporte con 8 configuraciones opcionales?

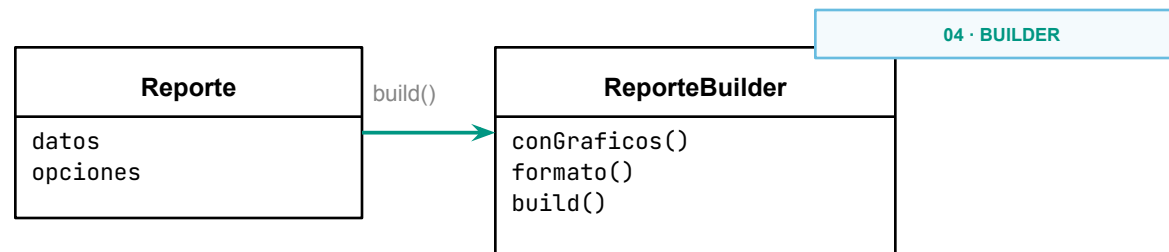
# Builder: solución, UML y ejemplo

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

- 1. ReporteBuilder guarda configuración parcial.
- 2. Cada método ajusta una opción y devuelve this.
- 3. build() valida y entrega el Reporte final.

## Participantes

- **Producto:** Reporte.
- **Builder:** interfaz de pasos.
- **Builder concreto:** acumula opciones.
- **Director:** opcional; define recetas comunes.



```
reporte_builder.ts
const reporte = new ReporteBuilder("ventas")
    .conGraficos(true)
    .formato("PDF")
    .paginas(30)
    .build();
```

Lectura: cada línea nombra una decisión; build() es el punto donde el objeto queda cerrado y validado.

**Director opcional:** puede encapsular recetas como reporteMensual() o reporteAuditoria().

## Evaluación

- **Ventaja:** llamadas legibles y configuración progresiva.
- **Ventaja:** separa construcción de representación final.
- **Riesgo:** builder que permite estados inválidos hasta build().
- **Usar cuando:** muchos parámetros opcionales o pasos con orden.
- **Evitar cuando:** dos o tres campos bastan.

**Comprobación:** ¿qué validación debería vivir en build() antes de entregar un Reporte?

## Diferencia que suele confundirse

| Pregunta          | Builder                           | Abstract Factory                       |
|-------------------|-----------------------------------|----------------------------------------|
| ¿Qué se crea?     | Un objeto complejo paso a paso    | Una familia de objetos compatibles     |
| ¿Qué varía?       | Opciones y orden de construcción  | La familia o proveedor completo        |
| ¿Cuándo elegirlo? | Muchas configuraciones opcionales | Combinaciones que deben ser coherentes |

**Error frecuente:** usar Builder para ocultar que faltan datos obligatorios. Si algo es obligatorio, que la API lo exija desde el inicio.

**En una oración:** permite crear objetos clonando un prototipo en lugar de construirlos desde cero.

**Analogía:** fotocopiar una plantilla y luego editar solo el nombre; no vuelves a dibujar todo el documento.

**Cuándo copiar conviene:** crear desde cero es costoso, hay muchas variantes parecidas o el objeto ya pasó por validaciones difíciles de repetir.

05 · PROTOTYPE

```
variantes.ts

const base = cargarPlantillaCostosa();
// Se necesitan 20 variantes casi iguales
// ¿repetir toda la configuración 20 veces?
```

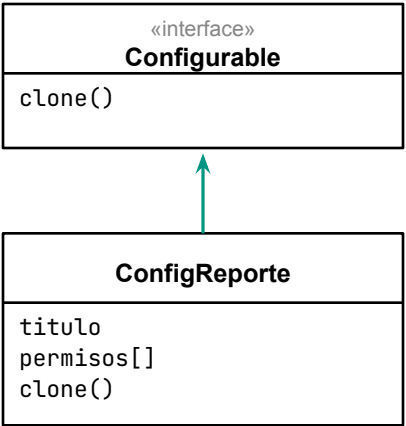
## Problema

- Reconstruir cada variante repite trabajo costoso.
- Copiar a mano campo por campo es frágil.
- Si el objeto tiene referencias internas, copiar mal comparte estado sin querer.

**Anticipa:** ¿qué debería copiarse y qué debería compartirse al clonar una configuración?



- 1. La interfaz declara clone().
- 2. Cada clase concreta sabe copiarse.
- 3. El cliente clona el prototipo y ajusta la variante.



clone() devuelve otra instancia del mismo tipo.

05 · PROTOTYPE

### Decisión crítica

| Tipo de copia | Qué copia                                        | Riesgo                                  |
|---------------|--------------------------------------------------|-----------------------------------------|
| Superficial   | Campos primitivos; comparte referencias internas | Un clone modifica la lista del original |
| Profunda      | También copia objetos internos                   | Más costo; exige copiar todo el grafo   |

**Regla práctica:** si el clone y el original no deben afectarse, copia profunda; si comparten recursos inmutables de forma deliberada, superficial puede bastar.

**Riesgo:** referencias internas compartidas sin documentar.

● ● ● prototype.ts

```
class ConfigReporte {
  constructor(
    public titulo: string,
    public permisos: string[]
  ) {}
  clone(): ConfigReporte {
    return new ConfigReporte(
      this.titulo,
      [...this.permisos] // copia de la lista
    );
  }
}

const base = new ConfigReporte("mensual", ["admin"]);
const copia = base.clone();
copia.titulo = "mensual-sucursal";
```

Resultado esperado: copia cambia el título sin alterar base; permisos se copia como lista nueva.

05 · PROTOTYPE

## Evaluación

- **Ventaja:** evita repetir construcción costosa.
- **Ventaja:** crea variantes desde un estado ya validado.
- **Riesgo:** confundir copia superficial con profunda.
- **Usar cuando:** copiar es más claro o barato que reconstruir.
- **Vs Builder:** Builder ensambla; Prototype duplica.

**Comprobación:** si permisos fuera un objeto con métodos, ¿alcanza con [...this.permisos]?

# Comparación de patrones

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

| Patrón           | Propósito principal                        | Problema que resuelve                      | Ejemplo de aplicación                 | Ventaja principal                         | Riesgo o desventaja                   |
|------------------|--------------------------------------------|--------------------------------------------|---------------------------------------|-------------------------------------------|---------------------------------------|
| Factory Method   | Crear un producto mediante especialización | if dispersos con new y clases concretas    | Notificaciones por canal              | Agregar variantes sin tocar el cliente    | Más clases e indirección              |
| Abstract Factory | Crear familias de productos relacionados   | Combinar piezas incompatibles              | UI para Windows / Linux               | Garantiza compatibilidad de familia       | Difícil agregar nuevos productos      |
| Singleton        | Controlar cantidad de instancias           | Estados duplicados que deberían ser únicos | Configuración o registro central      | Un punto de acceso controlado             | Estado global, pruebas y concurrencia |
| Builder          | Construir objetos complejos paso a paso    | Constructores con demasiados parámetros    | Reportes o pedidos configurables      | Llamadas legibles y validación en build() | API grande si se abusa                |
| Prototype        | Crear objetos mediante clonación           | Reconstruir configuraciones costosas       | Plantillas de documentos o personajes | Evita repetir inicialización costosa      | Copia superficial mal entendida       |

Leer por problema, no por nombre: un mismo sistema puede usar varios patrones en puntos distintos sin contradecirse.

## 1. ¿Qué patrón usarías para crear diferentes tipos de notificaciones?

- Factory Method: cada creador especializado decide el canal.

## 2. ¿Qué patrón usarías para construir un reporte con muchas configuraciones opcionales?

- Builder: acumula opciones y valida en build().

## 3. ¿Qué patrón permitiría crear una copia de una configuración existente?

- Prototype: clona una instancia preparada.

## 4. ¿Qué patrón permitiría trabajar con familias de componentes compatibles?

- Abstract Factory: entrega botón, ventana y menú de la misma familia.

## 5. ¿En qué situación Singleton podría generar más problemas que beneficios?

- Cuando solo oculta una variable global o cuando las pruebas necesitan estados independientes.

## Escenario 1 · Pagos

**Problema:** Una tienda agrega métodos de pago cada trimestre. Hoy Checkout tiene un switch con `new PagoTarjeta`, `new PagoPayPal` y `new PagoTransferencia`.

**Decisión:** Respuesta sugerida: Factory Method. Cada medio de pago obtiene un creador especializado y Checkout depende de la abstracción. Singleton no resuelve variantes; Builder no decide tipo de pago.

## Escenario 2 · UI multiplataforma

**Problema:** Una app debe mostrar botones, menús y ventanas coherentes en Android y web. El error reportado es mezclar un menú web con una ventana Android.

**Decisión:** Respuesta sugerida: Abstract Factory. Una fábrica por plataforma entrega la familia completa compatible. Factory Method alcanzaría para un solo componente, no para coordinar la familia.

## Escenario 3 · Reportes

**Problema:** Un reporte tiene nombre obligatorio y nueve opciones: gráficos, formato, paginación, filtros, firma, portada, anexos, compresión y destinatarios.

**Decisión:** Respuesta sugerida: Builder. Las opciones se leen paso a paso y build() valida combinaciones. Abstract Factory no encaja porque no hay familias incompatibles; Prototype serviría solo si ya existe un reporte base válido.

## Escenario 4 · Plantillas

**Problema:** Crear una ficha de personaje exige calcular atributos y validar reglas. Luego se necesitan 30 personajes casi iguales con cambios mínimos.

**Decisión:** Respuesta sugerida: Prototype. Clonar una ficha validada evita repetir el cálculo. La decisión delicada es copia profunda de inventario y habilidades. Builder construiría desde cero; no aprovecha la ficha ya validada.

## Checklist de decisión

- **Frase mínima:** “Elegí \_\_\_\_ porque el problema es \_\_\_\_; acepto el costo \_\_\_\_ y evito el riesgo \_\_\_\_.”
- **Variantes de un producto:** Factory Method.
- **Familia compatible:** Abstract Factory.
- **Una sola instancia real:** Singleton, con cuidado.
- **Muchas opciones legibles:** Builder.
- **Copiar estado validado:** Prototype.

**Señal de salida:** si eliminar el patrón no cambia ningún riesgo, probablemente sobra.

## Errores frecuentes al implementar

- Usar un patrón porque “se ve profesional”.
- Convertir cada new en una fábrica.
- Hacer Singleton de todo lo compartido.
- Builder con datos obligatorios escondidos.
- Prototype sin decidir copia superficial/profunda.

**Responsabilidad técnica:** documentar por qué se eligió el patrón y qué alternativa se descartó.

*[Insertar aquí la miniatura del video]*

Enlace: **[Insertar aquí el enlace del video de YouTube]**

## Qué observar durante el video

- Qué problema de creación aparece antes de nombrar el patrón.
- Qué participantes asumen la creación del objeto.
- Qué costo o límite menciona el ejemplo.

## Después del video:

1. ¿Qué problema de creación de objetos se mostró?
2. ¿Qué diferencias identificaste entre los patrones explicados?



- 1. Los patrones creacionales organizan y controlan la creación de objetos.
- 2. Cada patrón responde a un problema diferente; no hay uno universalmente mejor.
- 3. Factory Method delega una variante; Abstract Factory coordina una familia.
- 4. Singleton solo se justifica cuando la unicidad es real y controlada.
- 5. Builder hace legible una construcción con muchas opciones.
- 6. Prototype reutiliza una instancia validada y exige decidir el tipo de copia.
- 7. Bien aplicados reducen acoplamiento; mal aplicados aumentan complejidad.
- 8. La responsabilidad profesional implica evaluar consecuencias antes de implementar.

**Frase para llevarse: separar “usar un objeto” de “decidir cómo se crea” solo vale la pena cuando reduce un riesgo concreto.**

## Espacio abierto para preguntas.

Los estudiantes formulan sus propias dudas sobre los patrones, los diagramas o los ejemplos de código.

---

*Si hace falta iniciar la conversación: “¿qué diferencia todavía no está clara?”*

# Recordatorio: foro semanal

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

## Participa en el foro semanal

*Tema, instrucciones y fecha límite se publican en la plataforma del curso.*

ANÁLISIS Y DISEÑO DE SISTEMAS 2 · PATRONES CREACIONALES

# ¡Gracias por su atención!

---

Nos vemos en la próxima sesión.

El material de la clase y el foro semanal están disponibles en la plataforma del curso.